PANTHORA CLOTHING — E-COMMERCE SYSTEM

Full Technical Architecture & Code Reference Manual | Comprehensive Presentation Guide

1. Executive Summary & Full Technology Breakdown

Panthora Clothing is a full-stack e-commerce application engineered for a modern streetwear brand. Built on the **MERN Stack** (MongoDB, Express.js, React.js, Node.js), it integrates a responsive dark-themed user interface, global state management via Redux Toolkit, RESTful backend APIs, dual-mode database operations (MongoDB + In-Memory Fallback Store), and static local image serving for high-resolution clothing assets.

Layer / Subsystem	Framework / Package	Detailed Function & Usage
Frontend Core	React 18 + Vite 5	Single Page Application (SPA) container with high-performance hot module replacement.
UI Design System	Tailwind CSS 3	Utility-first styling for dark glassmorphism, responsive grid system, and custom typography.
Icons & Notifications	React Icons + React Hot Toast	Renders streetwear UI icons (HiOutlineShoppingBag, etc.) and toast notifications.
Client State	Redux Toolkit (@reduxjs/toolkit)	Centralized state management for shopping cart and user authentication with localStorage persistence.
Client Routing	React Router DOM 6	Declarative route handling for pages (/products, /product/:id, /cart, /checkout, /admin).
HTTP Client	Axios	Asynchronous HTTP requests with pre-configured base URL and credentials handling.
Backend Runtime	Node.js + Express.js 4	Asynchronous event-driven web server hosting REST API endpoints and static assets.
Database ORM	Mongoose 8	Object Data Modeling (ODM) layer for schema validation and MongoDB interaction.
Database Engine	MongoDB + MongoMemoryServer	Dual-mode setup: connects to local/cloud MongoDB with automatic in-memory MongoDB fallback.
Security & Auth	JSON Web Tokens (JWT) + bcryptjs	Password hashing pre-save hooks and signed JWT bearer tokens for secure user authentication.
Asset Delivery	Express Static Middleware	Direct file streaming of uploaded clothing photos via app.use('/uploads', ...).

2. System Architecture & Component Interaction Flow

The application follows a clean **Client-Server Architecture**. The React frontend operates independently on port 5173, while the Express server runs on port 5000. Communication occurs via JSON REST APIs over HTTP.

End-to-End Data Execution Lifecycle:

1. **User Action:** Customer opens category *Hoodies* on the frontend UI.
2. **React State Trigger:** Products.jsx triggers an asynchronous Axios call: api.get('/products?category=hoodie').
3. **Express Middleware Chain:** Request arrives at Express on port 5000. CORS middleware validates origin, body-parser parses query parameters.
4. **Router Dispatch:** Request hits productRoutes.js, invoking productController.getProducts.
5. **Database Execution:** Controller checks database connection status. If MongoDB is active, it runs Product.find({ category: 'hoodie' }). If offline, it queries store.getProducts().

6. **JSON Response:** Controller returns HTTP 200 with JSON payload containing array of products, prices, stock, and local image paths (e.g. /uploads/black-hoodie.png).

7. **React Render:** Component receives data, maps items, and renders ProductCard components. Image elements fetch photos directly from Vite/Express static upload endpoint.

3. Complete Codebase Directory & File Inventory

File Path	Module Type	Core Responsibilities & Implementation Details
server/server.js	Backend Bootstrap	Initializes Express server, sets up CORS, registers middleware, mounts static /uploads route, starts listener.
server/config/db.js	Database Config	Connects to MongoDB, manages auto-failover to in-memory Mongo, purges obsolete images, seeds updated product dataset.
server/models/Product.js	Mongoose Schema	Defines data schema for clothing products: name, description, price, category, sizes, colors, stock, images array.
server/models/User.js	Mongoose Schema	User account schema with pre-save bcrypt password hashing hook and instance method for password comparison.
server/models/Order.js	Mongoose Schema	Order schema storing customer ID, ordered items array, shipping address, total price, order status, payment method.
server/controllers/productController.js	Controller Logic	Functions for listing products with filtering/sorting, fetching product by ID, creating, updating, and deleting products.
server/controllers/authController.js	Controller Logic	Handles user registration, login verification, token generation, profile retrieval, and logout.
server/controllers/orderController.js	Controller Logic	Handles order creation, fetching user-specific order history, admin listing of all orders, and order status updates.
server/middleware/authMiddleware.js	Security Middleware	Extracts JWT bearer token from cookies/headers, verifies signature, populates req.user, checks admin role permissions.
server/middleware/errorHandler.js	Error Handler	Centralized error handling middleware formatting error JSON responses with appropriate HTTP status codes.
server/store.js	In-Memory Store	Class-based memory store (MemoryStore) holding fallback arrays for products, users, and orders.
server/seed.js	CLI Seed Script	Standalone command-line script to wipe MongoDB collections and insert sample clothing products and admin account.
client/src/App.jsx	Frontend Container	Main React Router container mounting Navbar, Footer, Toast notifications, and defining all page routes.
client/src/api/axios.js	HTTP Client Config	Configures Axios instance with base URL http://localhost:5000/api and withCredentials: true.
client/src/store/index.js	Redux Config	Configures global Redux store combining cartSlice and authSlice reducers.
client/src/store/slices/cartSlice.js	Redux Cart Slice	Redux state for shopping cart items, quantity modification, price calculations, and localStorage sync.
client/src/components/ProductCard.jsx	React Component	Product UI card with image preview, category badge, title, price, and Quick Add button.
client/src/pages/Home.jsx	React Page	Landing Page displaying hero banner, brand ethos, category spotlight, and featured clothing drop grid.
client/src/pages/Products.jsx	React Page	Catalog search & filter page with category sidebar, price sorting dropdown, and product grid.
client/src/pages/ProductDetail.jsx	React Page	Individual product showcase with image gallery, size selector, quantity counter, and Add to Cart action.
client/src/pages/Cart.jsx	React Page	Full shopping cart overview showing item list, price breakdown, promo code input, and proceed to checkout button.
client/src/pages/Checkout.jsx	React Page	Checkout page collecting customer shipping address, order summary, COD payment, and order submission.

4. Backend Code Deep-Dive & Line-by-Line Breakdown

4.1 Server Bootstrapper (server/server.js)

This file starts the Node.js process, binds middleware, configures static file serving, and connects routes.

```
const express = require('express');
const cors = require('cors');
const cookieParser = require('cookie-parser');
const path = require('path');
const connectDB = require('./config/db');
const errorHandler = require('./middleware/errorHandler');

const app = express();
const PORT = process.env.PORT || 5000;

// Middleware setup
app.use(cors({ origin: 'http://localhost:5173', credentials: true }));
app.use(express.json());
app.use(cookieParser());

// Serve local uploaded images statically at /uploads URL path
app.use('/uploads', express.static(path.join(__dirname, 'uploads')));

// API Route Mounting
app.use('/api/auth', require('./routes/authRoutes'));
app.use('/api/products', require('./routes/productRoutes'));
app.use('/api/orders', require('./routes/orderRoutes'));

app.use(errorHandler);

app.listen(PORT, () => console.log(`Panthora API server running on port ${PORT}`));
connectDB();
```

4.2 Resilient Database Connection & Auto-Seeding (server/config/db.js)

Connects to MongoDB; if local MongoDB is not running, seamlessly launches an in-memory MongoDB instance and auto-seeds local clothing products.

```
const mongoose = require('mongoose');
const Product = require('../models/Product');

const autoSeed = async () => {
  // Purge any old products with external unsplash URLs
  await Product.deleteMany({ 'images.0': { $regex: 'unsplash' } });

  const count = await Product.countDocuments();
  if (count === 0) {
    console.log('Seeding updated Panthora clothing products...');
    await Product.insertMany([
      { name: 'Black Oversized Hoodie', price: 3499, category: 'hoodie', images: ['/uploads/black-hoodie.png'] },
      { name: 'Printed Oversized Tee', price: 1799, category: 'tee', images: ['/uploads/printed-oversized-tee.png']
},
      { name: 'Black Trackpants', price: 2299, category: 'lower', images: ['/uploads/black-trackpants.png'] }
    ]);
  }
};
```

4.3 Product Controller API Logic (server/controllers/productController.js)

Processes incoming product catalog requests, applies category filters (hoodie, tee, lower), and handles price sorting.

```
const getProducts = async (req, res) => {
  try {
    const { category, sort, featured } = req.query;
    const filter = {};
    if (category && ['hoodie', 'tee', 'lower'].includes(category)) {
      filter.category = category;
    }
    if (featured === 'true') filter.isFeatured = true;

    let query = Product.find(filter);
    if (sort === 'price_asc') query = query.sort({ price: 1 });
    else if (sort === 'price_desc') query = query.sort({ price: -1 });
    else query = query.sort({ createdAt: -1 });

    const products = await query;
    res.json({ success: true, count: products.length, products });
  } catch (error) {
    res.status(500).json({ success: false, message: error.message });
  }
};
```

5. Frontend React & Redux Implementation Breakdown

5.1 Global Cart State Management (client/src/store/slices/cartSlice.js)

Manages adding items, calculating subtotal, increasing/decreasing quantities, and persisting to browser localStorage.

```
import { createSlice } from '@reduxjs/toolkit';

const savedCart = JSON.parse(localStorage.getItem('panthora_cart') || '[]');

const cartSlice = createSlice({
  name: 'cart',
  initialState: { items: savedCart },
  reducers: {
    addToCart: (state, action) => {
      const item = action.payload;
      const existing = state.items.find(i => i._id === item._id && i.selectedSize === item.selectedSize);
      if (existing) {
        existing.quantity += item.quantity || 1;
      } else {
        state.items.push({ ...item, quantity: item.quantity || 1 });
      }
      localStorage.setItem('panthora_cart', JSON.stringify(state.items));
    },
    removeFromCart: (state, action) => {
      state.items = state.items.filter(i => i._id !== action.payload);
      localStorage.setItem('panthora_cart', JSON.stringify(state.items));
    }
  }
});
export const { addToCart, removeFromCart } = cartSlice.actions;
export default cartSlice.reducer;
```

5.2 Product Card Component (client/src/components/ProductCard.jsx)

Reusable UI card rendering product image, category pill, name, formatted price, and dispatching Add to Cart.

```
import { useDispatch } from 'react-redux';
import { addToCart } from '../store/slices/cartSlice';
import { toast } from 'react-hot-toast';

export default function ProductCard({ product }) {
  const dispatch = useDispatch();
  const handleAdd = (e) => {
    e.preventDefault();
    dispatch(addToCart(product));
    toast.success(`${product.name} added to cart!`);
  };
  return (
    <div className="card group relative overflow-hidden rounded-xl bg-bg-surface border border-border">
      <img src={product.images?.[0]} alt={product.name} className="w-full h-80 object-cover" />
      <div className="p-4">
        <span className="text-xs text-accent uppercase font-bold">{product.category}</span>
        <h3 className="font-bold text-lg text-text-primary">{product.name}</h3>
        <p className="text-accent font-semibold">Rs. {product.price}</p>
        <button onClick={handleAdd} className="btn-primary w-full mt-2">
          Add to Cart
        </button>
      </div>
    </div>
  );
}
```

6. Category & Image Mapping Table

All products are classified into 3 major categories. Each product points to an actual file in /uploads/:

Category	Product Title	Price	Image Filename	Preview
Hoodies	Black Oversized Hoodie	Rs. 3,499	black-hoodie.png	
Hoodies	Zipper Hoodie	Rs. 3,999	zipper-hoodie.png	Yes
Hoodies	Grey Pullover Hoodie	Rs. 2,999	grey-hoodie.png	Yes
Hoodies	Brown Heavyweight Hoodie	Rs. 3,799	brown-hoodie.png	Yes
Tees	Black Oversized Tee	Rs. 1,499	black-oversized-tee.png	Yes
Tees	Printed Oversized Tee	Rs. 1,799	printed-oversized-tee.png	
Tees	Acid Wash Oversized Tee	Rs. 2,199	acid-wash-oversized-tee.png	Yes
Tees	Full Sleeve Printed Tee	Rs. 1,999	full-sleeve-printed-tee.png	Yes
Lowers	Black Joggers	Rs. 2,499	black-joggers.png	Yes
Lowers	Black Trackpants	Rs. 2,299	black-trackpants.png	
Lowers	Green Trackpants	Rs. 2,299	green-trackpants.png	Yes
Lowers	White Trackpants	Rs. 2,399	white-trackpants.png	Yes

7. Mentor / Presentation Viva Q&A; Reference Guide

Use these answers when presenting this project to your teacher or evaluator:

Q1. What architecture does Panthora Clothing use?

Ans: Panthora Clothing uses a MERN (MongoDB, Express, React, Node) Client-Server Architecture. React handles the SPA frontend on port 5173, Express handles REST APIs on port 5000, and MongoDB manages product/order data.

Q2. How is offline or database fallback handled?

Ans: In server/config/db.js, if local MongoDB connection fails, the server automatically spawns an in-memory MongoDB database (MongoMemoryServer) and populates initial clothing products so the site always functions seamlessly.

Q3. How are images uploaded and served to users?

Ans: Images uploaded in 'Clothes photos' are synced into /public/uploads/ and server/uploads/. Express serves them via app.use('/uploads', express.static(...)), and React displays them via standard image paths.

Q4. How does cart state persist on page reload?

Ans: Redux Toolkit (cartSlice.js) maintains cart state in Redux memory and synchronizes every state change to browser localStorage. On page load, Redux initializes state from localStorage.